

NAME: HIKMAT ARIIKE ABDULAZEEZ

Implementing TLS, SSO, and Vault with Keycloak

1. Showing keycloak is already installed

```
hikmat@hikmat-VMware-Virtual-Platform:~$ tar -xzf keycloak-26.3.4.tar.gz
hikmat@hikmat-VMware-Virtual-Platform:~/keycloak-26.3.4$ ls bin/kc.sh
```

2. Creating a self-signed cert (with SAN) inside the Keycloak folder to work for <https://localhost>.

```
hikmat@hikmat-VMware-Virtual-Platform:~/keycloak-26.3.4$ ls bin/kc.sh
bin/kc.sh
hikmat@hikmat-VMware-Virtual-Platform:~/keycloak-26.3.4$ mkdir -p ~/keycloak-26.3.4/certs
openssl req -x509 -newkey rsa:2048 -nodes -days 365 \
  -keyout ~/keycloak-26.3.4/certs/keycloak.key \
  -out ~/keycloak-26.3.4/certs/keycloak.crt \
  -subj "/CN=localhost" \
  -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
```

3. Configuration forces HTTPS

```
hikmat@hikmat-VMware-Virtual-Platform:~/keycloak-26.3.4$ ls ~/keycloak-26.3.4/certs/
keycloak.crt  keycloak.key
hikmat@hikmat-VMware-Virtual-Platform:~/keycloak-26.3.4$ mkdir -p ~/keycloak-26.3.4/conf
cat > ~/keycloak-26.3.4/conf/keycloak.conf <<'CONF'
https-port=8443
http-enabled=false
https-certificate-file=${kc.home.dir}/certs/keycloak.crt
https-certificate-key-file=${kc.home.dir}/certs/keycloak.key
hostname=localhost
hostname-strict=false
CONF
```

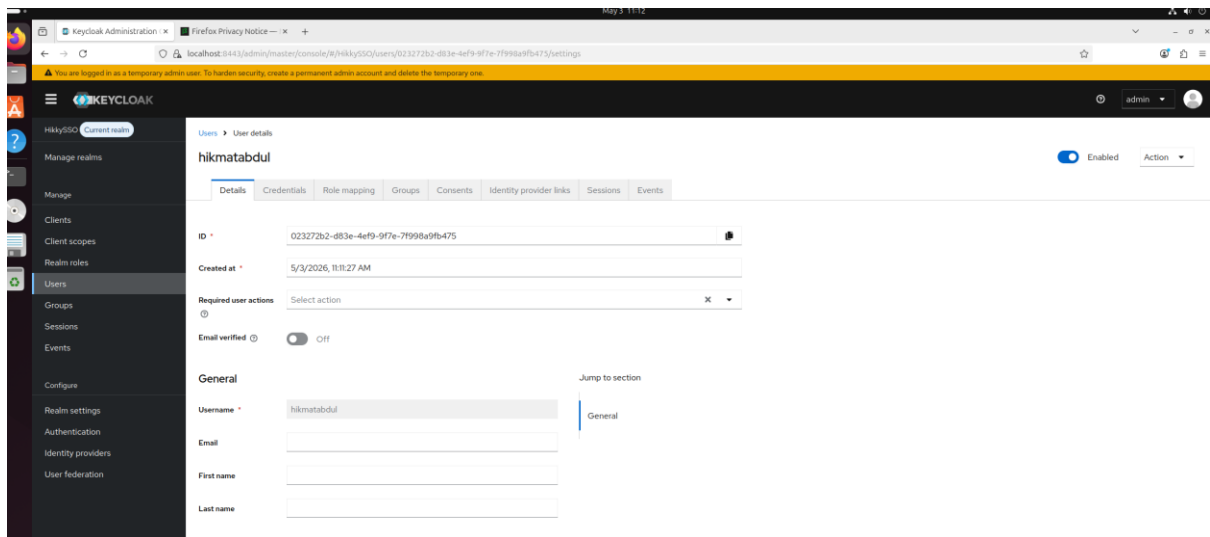
4. Starting Keycloak

```
hikmat@hikmat-VMware-Virtual-Platform:~/keycloak-26.3.4$ cd ~/keycloak-26.3.4
bin/kc.sh start-dev
Updating the configuration and installing your custom providers, if any. Please wait.
2026-05-03 10:28:53,200 INFO [io.quarkus.deployment.QuarkusAugmentor] (main) Quarkus augmentation completed in 22502ms
Running the server in development mode. DO NOT use this configuration in production.
2026-05-03 10:28:58,990 INFO [org.keycloak.url.HostnameV2ProviderFactory] (main) If hostname is specified, hostname-strict is effectively i
2026-05-03 10:29:07,449 INFO [org.keycloak.quarkus.runtime.storage.database.liquibase.QuarkusJpaUpdaterProvider] (main) Initializing databa
2026-05-03 10:29:13,631 INFO [org.keycloak.spi.infinispan.impl.embedded.JGroupsConfigurator] (main) JGroups JDBC_PING discovery enabled.
2026-05-03 10:29:13,774 INFO [org.infinispan.CONTAINER] (main) Virtual threads support enabled
2026-05-03 10:29:13,965 INFO [org.infinispan.CONTAINER] (main) ISPN000556: Starting user marshaller 'org.infinispan.commons.marshall.Immuta
2026-05-03 10:29:14,375 INFO [org.keycloak.connections.infinispan.DefaultInfinispanConnectionProviderFactory] (main) Node name: node_230029
2026-05-03 10:29:14,530 INFO [org.keycloak.services] (main) KC-SERVICES0050: Initializing master realm
2026-05-03 10:29:17,067 WARN [org.keycloak.services] (main) KC-SERVICES0110: Environment variable 'KEYCLOAK_ADMIN' is deprecated, use 'KC_E
2026-05-03 10:29:17,069 WARN [org.keycloak.services] (main) KC-SERVICES0110: Environment variable 'KEYCLOAK_ADMIN_PASSWORD' is deprecated,
2026-05-03 10:29:17,294 INFO [org.keycloak.services] (main) KC-SERVICES0077: Created temporary admin user with username admin
2026-05-03 10:29:17,583 INFO [io.quarkus] (main) Keycloak 26.3.4 on JVM (powered by Quarkus 3.20.2.2) started in 23.987s. Listening on: htt
2026-05-03 10:29:17,586 INFO [io.quarkus] (main) Profile dev activated.
2026-05-03 10:29:17,586 INFO [io.quarkus] (main) Installed features: [agroal, cdi, hibernate-orm, jdbc-h2, keycloak, narayana-jta, opentele
```

5. Keycloak listening on <https://localhost:8443>

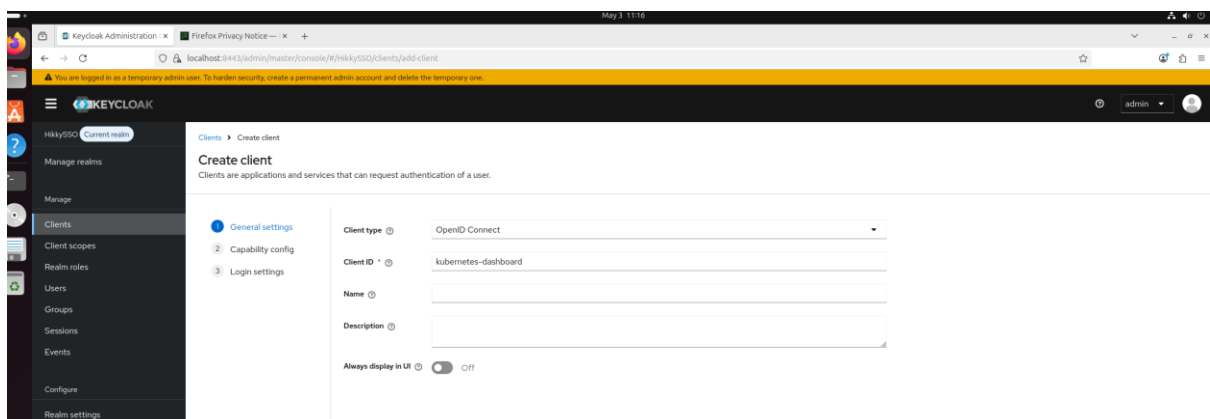
SSO

6. Creating Users

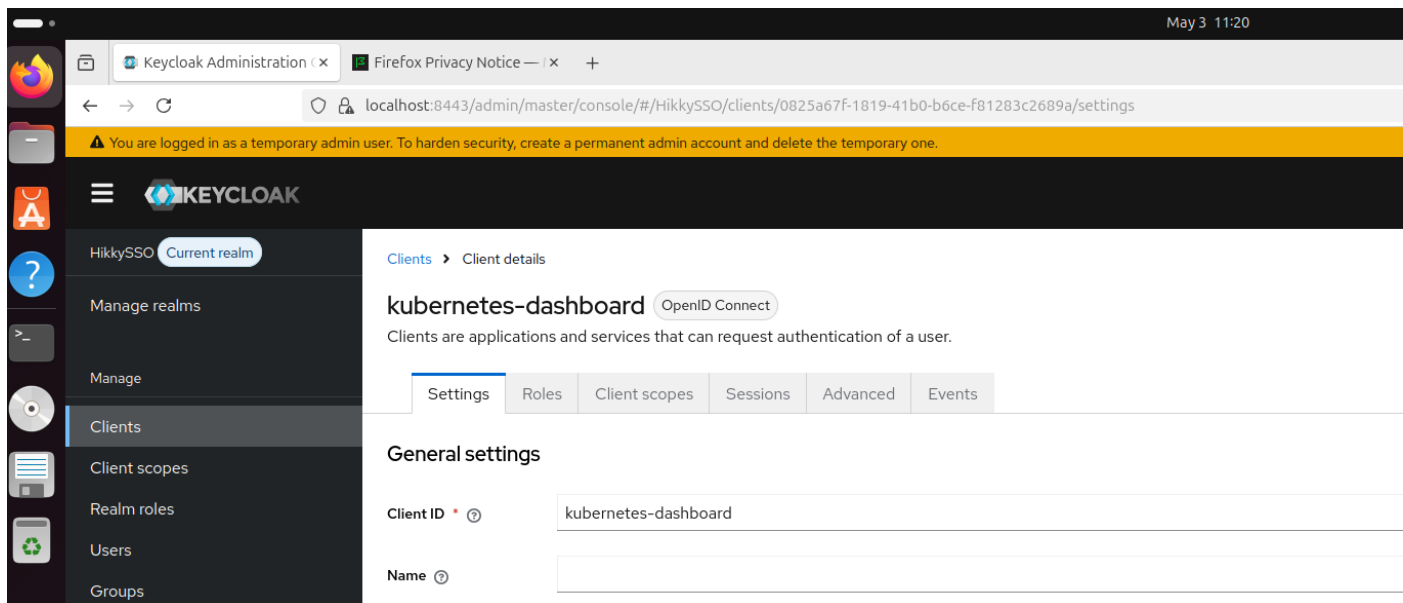
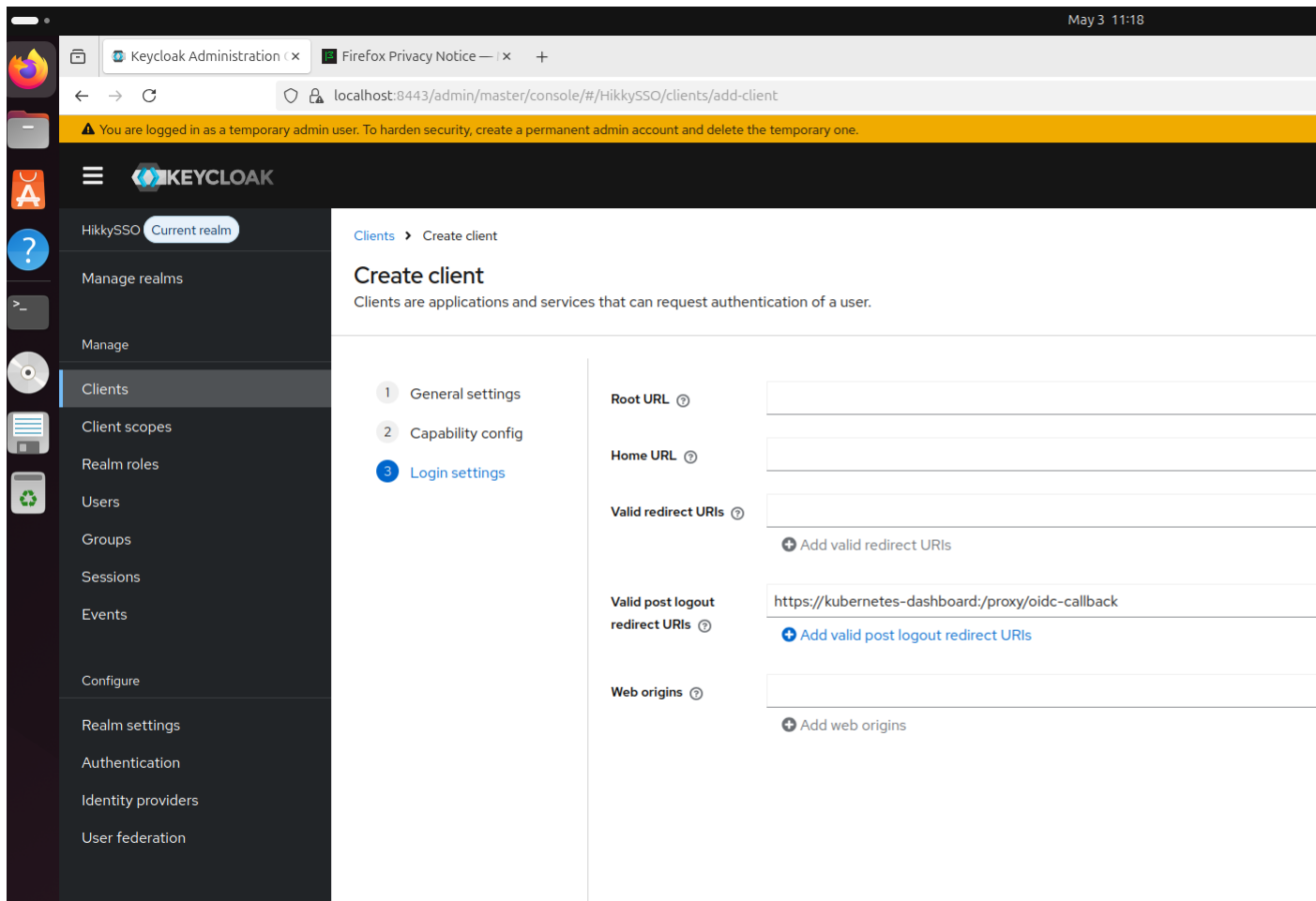


The screenshot shows the Keycloak administration interface. The left sidebar contains a menu with options like 'Manage realms', 'Clients', 'Groups', 'Sessions', 'Events', 'Configure', 'Realm settings', 'Authentication', 'Identity providers', and 'User federation'. The main content area is titled 'Users > User details' and shows the details for a user named 'hikmatabdul'. The user is currently 'Enabled'. The 'Details' tab is selected, showing fields for 'ID' (023272b2-d83e-4ef9-9f7e-7f998a9fb475), 'Created at' (5/3/2026, 11:11:27 AM), 'Required user actions' (Select action), and 'Email verified' (Off). Below this, the 'General' tab is selected, showing fields for 'Username' (hikmatabdul), 'Email', 'First name', and 'Last name'. A 'Jump to section' link is also visible.

7. Creating OIDC Client



The screenshot shows the Keycloak administration interface. The left sidebar is the same as in the previous screenshot. The main content area is titled 'Clients > Create client' and shows the 'Create client' form. The form has three tabs: 'General settings', 'Capability config', and 'Login settings'. The 'General settings' tab is selected, showing fields for 'Client type' (OpenID Connect), 'Client ID' (kubernetes-dashboard), 'Name', 'Description', and 'Always display in UI' (Off).



8. Preparing Kubernetes MikroK8s + Dashboard

```

hikmat@hikmat-VMware-Virtual-Platform:~/keycloak-26.3.4$ sudo snap install microk8s --classic --channel=1.30/stable
sudo usermod -aG microk8s $USER
newgrp microk8s
[sudo] password for hikmat:
Sorry, try again.
[sudo] password for hikmat:
Ensure prerequisites for "microk8s" are available
Download snap "core20" (2769) from channel "stable"
Download snap "core20" (2769) from channel "stable"
Download snap "core20" (2769) from channel "stable"
microk8s (1.30/stable) v1.30.14 from Canonical✓ installed
hikmat@hikmat-VMware-Virtual-Platform:~/keycloak-26.3.4$

```

```

hikmat@hikmat-VMware-Virtual-Platform:~/keycloak-26.3.4$ microk8s enable dashboard
microk8s kubectl get pods -A | grep dashboard
Infer repository core for addon dashboard
Enabling Kubernetes Dashboard
Infer repository core for addon metrics-server
Enabling Metrics-Server
serviceaccount/metrics-server created
clusterrole.rbac.authorization.k8s.io/system:aggregated-metrics-reader created
clusterrole.rbac.authorization.k8s.io/system:metrics-server created
rolebinding.rbac.authorization.k8s.io/metrics-server-auth-reader created
clusterrolebinding.rbac.authorization.k8s.io/metrics-server:system:auth-delegator created
clusterrolebinding.rbac.authorization.k8s.io/system:metrics-server created
service/metrics-server created
deployment.apps/metrics-server created
apiservice.apiregistration.k8s.io/v1beta1.metrics.k8s.io created
clusterrolebinding.rbac.authorization.k8s.io/microk8s-admin created
Metrics-Server is enabled
Applying manifest
serviceaccount/kubernetes-dashboard created
service/kubernetes-dashboard created
secret/kubernetes-dashboard-certs created
secret/kubernetes-dashboard-csrf created
secret/kubernetes-dashboard-key-holder created
configmap/kubernetes-dashboard-settings created
role.rbac.authorization.k8s.io/kubernetes-dashboard created
clusterrole.rbac.authorization.k8s.io/kubernetes-dashboard created
rolebinding.rbac.authorization.k8s.io/kubernetes-dashboard created
clusterrolebinding.rbac.authorization.k8s.io/kubernetes-dashboard created
deployment.apps/kubernetes-dashboard created
service/dashboard-metrics-scraper created
deployment.apps/dashboard-metrics-scraper created
secret/microk8s-dashboard-token created

If RBAC is not enabled access the dashboard using the token retrieved with:

microk8s kubectl describe secret -n kube-system microk8s-dashboard-token

Use this token in the https login UI of the kubernetes-dashboard service.

In an RBAC enabled setup (microk8s enable RBAC) you need to create a user with restricted
permissions as shown in:
https://github.com/kubernetes/dashboard/blob/master/docs/user/access-control/creating-sample-user.md

kube-system dashboard-metrics-scraper-795895d745-mddv8 0/1 ContainerCreating 0 18s
kube-system kubernetes-dashboard-6796797fb5-x77bz 0/1 ContainerCreating 0 21s
hikmat@hikmat-VMware-Virtual-Platform:~/keycloak-26.3.4$

```

9. Make Kubernetes API Server Trust Keycloak (OIDC)

```

hikmat@hikmat-VMware-Virtual-Platform:~/keycloak-26.3.4$ sudo snap install kubectl --classic
kubectl 1.35.4 from Canonical✓ installed
hikmat@hikmat-VMware-Virtual-Platform:~/keycloak-26.3.4$ sudo nano /var/snap/microk8s/current/args/kube-apiserver
hikmat@hikmat-VMware-Virtual-Platform:~/keycloak-26.3.4$

```

```
hikmat@hikmat-VMware-Virtual-Platform: ~/keycloak-26.3.4
$ nano /var/snap/microk8s/current/args/kube-apiserver
--cert-dir=$(SNAP_DATA)/certs
--service-cluster-ip-range=10.152.183.0/24
--authorization-mode=AlwaysAllow
--service-account-key-file=$(SNAP_DATA)/certs/serviceaccount.key
--client-ca-file=$(SNAP_DATA)/certs/ca.crt
--tls-cert-file=$(SNAP_DATA)/certs/server.crt
--tls-private-key-file=$(SNAP_DATA)/certs/server.key
--tls-cipher-suites=TLS_AES_128_GCM_SHA256,TLS_AES_256_GCM_SHA384,TLS_CHACHA20_POLY1305_SHA256,TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA,TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256,TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA,TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384
--kubelet-client-certificate=$(SNAP_DATA)/certs/apiserver-kubelet-client.crt
--kubelet-client-key=$(SNAP_DATA)/certs/apiserver-kubelet-client.key
--secure-port=6443
--etcd-servers="unix://$(SNAP_DATA)/var/kubernetes/backend/kine.sock:12379"
--allow-privileged=true
--service-account-issuer="https://kubernetes.default.svc"
--service-account-signing-key-file=$(SNAP_DATA)/certs/serviceaccount.key
--event-ttl=5m
--profiling=false

# Enable the aggregation layer
--requestheader-client-ca-file=$(SNAP_DATA)/certs/front-proxy-ca.crt
--requestheader-allowed-names=front-proxy-client
--requestheader-extra-headers-prefix=X-Remote-Extra-
--requestheader-group-headers=X-Remote-Group
--requestheader-username-headers=X-Remote-User
--proxy-client-cert-file=$(SNAP_DATA)/certs/front-proxy-client.crt
--proxy-client-key-file=$(SNAP_DATA)/certs/front-proxy-client.key
# Enable the admission layer
--enable-admission-plugins=EventedAdmissionLifecycleController
--admission-control-config-file=$(SNAP_DATA)/args/admission-control-config-file.yaml
--kubelet-certificate-authority=$(SNAP_DATA)/certs/ca.crt
--kubelet-preferred-address-types=InternalIP,Hostname,InternalDNS,ExternalDNS,ExternalIP
--oidc-issuer-url=https://localhost:8443/realms/Hikky550
--oidc-client-id=kubernetes-dashboard
--oidc-username-claim=email
--oidc-groups-claim=groups
--oidc-ca-file=/usr/local/share/ca-certificates/keycloak.crt
```

```
hikmat@hikmat-VMware-Virtual-Platform: ~/keycloak-26.3.4$ sudo nano /var/snap/microk8s/current/args/kube-apiserver
hikmat@hikmat-VMware-Virtual-Platform: ~/keycloak-26.3.4$ sudo snap restart microk8s
2026-05-03T11:50:21+03:00 INFO Waiting for "snap.microk8s.daemon-containerd.service" to stop.
2026-05-03T11:50:41+03:00 INFO Waiting for "snap.microk8s.daemon-k8s-dqlite.service" to stop.
Restarted.
hikmat@hikmat-VMware-Virtual-Platform: ~/keycloak-26.3.4$
```

10. Patch the Dashboard to Enable OIDC Authentication

```
hikmat@hikmat-VMware-Virtual-Platform: ~/keycloak-26.3.4$ microk8s kubectl -n kube-system patch deploy kubernetes-dashboard \
--type=json' -p='[
  {
    "op": "add",
    "path": "/spec/template/spec/volumes/-",
    "value": {
      "name": "keycloak-ca",
      "configMap": {
        "name": "keycloak-ca"
      }
    }
  },
  {
    "op": "add",
    "path": "/spec/template/spec/containers/0/volumeMounts/-",
    "value": {
      "name": "keycloak-ca",
      "mountPath": "/etc/keycloak",
      "readOnly": true
    }
  },
  {
    "op": "add",
    "path": "/spec/template/spec/containers/0/args/-",
    "value": "--authentication-mode=oidc"
  },
  {
    "op": "add",
    "path": "/spec/template/spec/containers/0/args/-",
    "value": "--oidc-issuer-url=https://localhost:8443/realms/SS0demo"
  },
  {
    "op": "add",
    "path": "/spec/template/spec/containers/0/args/-",
    "value": "--oidc-client-id=kubernetes-dashboard"
  },
  {
    "op": "add",
    "path": "/spec/template/spec/containers/0/args/-",
    "value": "--oidc-username-claim=email"
  },
  {
    "op": "add",
    "path": "/spec/template/spec/containers/0/args/-",
    "value": "--oidc-groups-claim=groups"
  },
  {
    "op": "add",
    "path": "/spec/template/spec/containers/0/args/-",
    "value": "--oidc-ca-file=/etc/keycloak/ca.crt"
  }
]'
deployment.apps/kubernetes-dashboard patched
```

11. Confirming the OIDC Authentication

```
hikmat@hikmat-VMware-Virtual-Platform: ~$ microk8s kubectl -n kube-system get deploy kubernetes-dashboard \
-o yaml | grep -A2 'authentication-mode'
- --authentication-mode=oidc
- --oidc-issuer-url=https://localhost:8443/realms/SS0demo
- --oidc-client-id=kubernetes-dashboard

hikmat@hikmat-VMware-Virtual-Platform: ~$ microk8s kubectl get pods -n kube-system | grep dashboard
dashboard-metrics-scraper-795895d745-mddv8      1/1      Running      2 (23m ago)    4h55m
kubernetes-dashboard-57dc4988d6-5ssg6          0/1      ContainerCreating      0              4h31m
kubernetes-dashboard-6796797fb5-x77bz          1/1      Running      2 (23m ago)    4h55m
hikmat@hikmat-VMware-Virtual-Platform: ~$
```

12. Testing the Kubernetes dashboard in Firefox

